FLYRANK FRONTEND AI ENGINEERING TRACK · FE-07 DELIVERABLE

Tool Results & Structured Output in the UI

Generative UI system rendering typed server-side tool execution, 4 distinct lifecycle states, rich component visualization, and designed error recovery boundaries.

CANDIDATE	ASSIGNMENT CODE	PRIMARY AI PLATFORM	STACK & ARCHITECTURE
KM Siam	**FE-07 (Week 5 · 5h)**	**Google Antigravity**	**Next.js 15 · Zod · React 19**

Executive Summary: Successfully implemented the `scorePromptEngineering` server-side tool with strict Zod schema validation, a deterministic execution engine, and a complete Generative UI lifecycle renderer (`ToolPartRenderer.jsx`). All four tool part states (*input-streaming, input-available, output-available, output-error*) render with distinct visual treatments and telemetry. Rich component rendering replaces raw JSON dumps with the interactive `PromptScoreCard` and designed `ToolErrorCard` . 47/47 automated tests passed and 14 routes compiled cleanly in Next.js 15.5.

1. Server-Side Tool Contract & Zod Specification

The tool is isolated in `src/lib/tools/promptAuditTool.js` and registered under the contract name `scorePromptEngineering` . It accepts prompt text, target domain, and threshold constraints, enforcing schema validation via Zod before invoking the deterministic execution function.

Zod Input Schema Definition

```
export const promptAuditInputSchema = z.object({
  promptText: z
    .string()
    .min(5, "Prompt must be at least 5 characters long")
    .max(4000, "Prompt cannot exceed 4000 characters"),
  targetDomain: z
    .enum(["coding", "compliance", "analysis", "general"])
    .default("general"),
  minimumScoreThreshold: z
    .number()
    .min(0)
    .max(100)
    .optional()
    .default(70),
});
```

Tool Contract Matrix

Property	Specification
Tool Name	`scorePromptEngineering`
Validator	`zod` v3.24.2
Execution Function	`executePromptAudit(args)`
Error Trigger	`TRIGGER_TOOL_ERROR` keyword
Heuristic Dimensions	Clarity (35%), Specificity (35%), Guardrails (30%)
Return Shape	Typed Object (score, breakdown, recommendations, tokens, latency)

2. Four Distinct Tool Part Lifecycle States

Instead of unformatted JSON blobs, `ToolPartRenderer.jsx` binds each state of the tool invocation lifecycle to a purpose-built visual treatment:

1. Input Streaming `RUNNING`
"What is it doing?" Pulsing radar icon with dynamic parameter counter, live token telemetry, and shimmering animated skeleton.

2. Input Available `READY`
"With what input?" Collapsible pill inspector displaying validated parameters, domain tag, threshold, and character count.

3. Output Available `SUCCESS`
"What came back?" Rich `PromptScoreCard` rendering composite score badge, dimension bars, SVG bar chart, and recommendations.

4. Output Error `FAILED`
"What went wrong?" Designed `ToolErrorCard` displaying error taxonomy, remediation steps, and an interactive Retry trigger.

3. Rich Component Visualization vs. Raw JSON Dump

Output-Available: PromptScoreCard.jsx

Renders structured analytical output as an enterprise scorecard with hand-crafted SVG dimension visualization and actionable checklists:

- **Composite Grade:** Tier pill (`EXCELLENT` , `GOOD` , `MODERATE` , `NEEDS WORK`) with color coded ring.
- **Dimensional Metrics:** Clarity (35%), Specificity (35%), and Guardrails (30%) progress indicators.
- **Hand-Rolled SVG Chart:** Responsive SVG bar chart showing dimension weights without bulky chart libraries.
- **Remediation Checklist:** Interactive bulleted recommendations for prompt optimization.
- **FinOps Telemetry:** Latency in milliseconds and calculated token count.

Output-Error: ToolErrorCard.jsx

Designed failure boundary providing deterministic error containment rather than white-screen crashes or unhandled promise rejections:

- **Error Taxonomy:** Distinct badge (e.g. `ERR_INSUFFICIENT_CONTEXT`).
- **User-Facing Remediation:** Explains exactly what was missing (e.g., prompt brevity, missing constraints) in clear human language.
- **Interactive Recovery:** Built-in "Retry Execution" button to re-trigger the pipeline with safe fallback parameters.
- **Safe Degradation:** Prevents unhandled crashes in client chat streams.

4. Interactive Generative UI Harness (/generative-ui)

An interactive demonstration harness was deployed at `/generative-ui` (mounted in `src/app/generative-ui/page.jsx` via `src/components/tools/ToolLifecycleDemo.jsx`) allowing evaluators to trigger both the successful end-to-end tool execution and the designed error boundary with real-time state visualization.

Tool Lifecycle Component State Machine

```
// Morphing tool state simulation & renderer
const runToolCycle = async (simulateFailure = false) => {
  setToolState("input-streaming");
  await sleep(650);

  setToolState("input-available");
  await sleep(550);

  if (simulateFailure) {
    const errorResult = await executePromptAudit({
      promptText: "TRIGGER_TOOL_ERROR: Missing context",
    });
    setToolOutput(errorResult);
    setToolState("output-error");
  } else {
    const result = await executePromptAudit({
      promptText: activePrompt,
      targetDomain: activeDomain,
      minimumScoreThreshold: 70,
    });
    setToolOutput(result);
    setToolState("output-available");
  }
};
```

Structured Output Return Shape

```
{
  "success": true,
  "overallScore": 86,
  "verdict": "EXCELLENT",
  "dimensions": {
    "clarity": 88,
    "specificity": 85,
    "guardrails": 85
  },
  "recommendations": [
    "Include explicit negative constraints (e.g., 'Do not output markdown comments')",
    "Specify maximum response token limits for deterministic latency"
  ],
  "telemetry": {
    "latencyMs": 42,
    "tokenCount": 54,
    "toolName": "scorePromptEngineering"
  }
}
```

5. Automated Test Suite & Next.js Build Verification

An automated test suite was engineered in `tests/tools.test.js` and added to `package.json`. The suite tests schema enforcement, return shape consistency, designed error containment, and state machine rendering:

Test Suite	Tests Passed	Key Assertions Verified
`tests/tools.test.js`	**6 / 6 Passed**	Validates Zod schema rejects short prompts; verifies return shape contract; tests error trigger taxonomy; tests 4 lifecycle states.
`tests/streaming.test.js`	**7 / 7 Passed**	Verifies token streaming, thinking handoff, abort controller, and auto-scroll pinning.
`tests/accessibility.test.js`	**10 / 10 Passed**	Verifies APG compliance for Modal, Tabs, Disclosure; focus trapping, roving tabindex, zero `any` in TypeScript.
`tests/routes.test.js`	**14 / 14 Passed**	Verifies all routes, responsive breakpoints (375px/1280px), health check data fetching, zero secrets.
`tests/promptcraft.test.js`	**10 / 10 Passed**	Verifies prompt template parsing, heuristic scoring, streaming simulator, FinOps token calculations.
Total Automated Suite (npm test)	**47 / 47 Passed (100%)**	**Zero failures, zero regressions across the complete repository.**

Build & Deployment Certification:

- `next build` : 14/14 static & dynamic routes compiled in 5.0s with 0 errors.
- TypeScript verification: `tsc --noEmit` compiled cleanly with 0 type errors.
- Live Preview URL: https://flyrank-internship-front-end-ai-engineering.vercel.app/generative-ui
- Vercel Git-connected deployment enabled: every commit to `main` triggers automated build and live preview deployment.

FlyRank Frontend AI Engineering Track · Candidate: KM Siam · AI Platform: Google Antigravity · MIT License © 2026